

Akshay Shivashankar Bharadwaj

Los Angeles, CA | ab79035@usc.edu/akshaysbharadwaj2k@gmail.com | +1 213 301 6674
ab79035.github.io/Portfolio | github.com/akshaysb2k | linkedin.com/in/akshay-s-bharadwaj2k

Education

- University of Southern California**, MS in Computer Science (GPA 3.86) Jan 2025 – Dec 2026
- **Courses:** Analysis and Design of Algorithms, Web Technologies, Operating Systems, Cloud and Distributed Systems
- BMS College of Engineering**, BE in Computer Science (GPA 4.0) Aug 2018 – Jun 2022
- **Courses:** Data Structures and Algorithms, Web and Database design, Artificial Intelligence, Machine Learning

Experience

- IT Engineer**, Qualcomm - Bengaluru, India June 2022 – Dec 2024
- Engineered a **ReactJS-Flask** internal tool integrating **ServiceNow APIs** to automate end-to-end RFC (request for change) workflows, eliminating **100+ engineering hours of manual effort per month**.
 - Built end-to-end containerized microservices infrastructure using **Docker**, **Kubernetes (EKS)**, and AWS CloudFormation (S3, RDS, Route53, VPC, Lambda), cutting deployment time by **50%** and eliminating environment-parity issues.
 - Drove **cross-functional collaboration** with Dev, QA, and Infra teams to implement **CI/CD pipelines** via Jenkins and CloudFormation, replacing manual deployments and reducing environment setup errors to **near-zero**.
 - Designed an **AWS CloudFormation automation** framework to onboard **70+ web applications** with MFA and 24/7 CloudWatch/DataDog alerting to enforce security and availability SLAs and leveraged **GenAI tools** to drive cloud cost optimization by over **60%**.
- Interim Intern**, Qualcomm - Bengaluru, India Feb 2022 – Jun 2022
- Built a **self-service AWS deployment infrastructure** using **CloudFormation and Lambda** that enabled teams to deploy Java applications with minimal manual input, cutting new environment setup from days to **under an hour**.

Projects

- Operating Systems Kernel Development: Weenix (Unix-Like OS Kernel)** Aug 2025 – Dec 2025
- Designed and implemented process and thread management in C, including scheduling, context switching, and synchronization.
 - Achieved full system call support by building a **virtual file system (VFS)** within a Linux-inspired OS kernel architecture.
 - Ensured robust memory isolation by implementing **virtual memory mapping** and **page fault handling** for concurrent user processes.
 - Supported concurrent process creation by developing `fork()` based child process creation in the Weenix kernel.
- Raft Consensus Protocol: Distributed Systems (C++20, gRPC)** Jan 2026 – May 2026
- Achieved **99.9% uptime** by implementing **Raft consensus leader election and log replication protocols** in C++20 using gRPC.
 - Guaranteed safety and liveness under node crashes, network partitions, and delayed RPC delivery across simulated failure scenarios.
 - Eliminated race conditions by designing concurrent, **thread-safe state management** for distributed RPC handlers.
 - Achieved **zero data loss** and **sub-500ms leader re-election** validated via fault-injection tests under real-world failure conditions.
- Artsy: Web & Mobile App using Django, PyTorch, Angular.js, Android Studio** Jan 2025 – May 2025
- Delivered **sub-200ms API response times** by building an **Angular.js** frontend with a **Django REST Framework** API and GCP deployment.
 - Achieved **87% accuracy** by training a **PyTorch** image classifier on artwork metadata scraped and normalized using **BeautifulSoup**.
 - Built an **Android Studio** mobile client consuming the REST APIs to deliver real-time artwork discovery on mobile.
- Eye-Move: Eye-Tracking Based Typing Application (Computer Vision)** Oct 2021 – Jun 2022
- Enabled hands-free communication for motor-neuron disability patients by building a virtual keyboard using **OpenCV and DLib**.
 - Achieved **87.5% typing accuracy** by developing precise blink detection algorithms calibrated for varied lighting conditions.
 - Reduced communication barriers for disabled users by optimizing detection latency and error thresholds in real-time.
- Moonquake Alert System (MAS), Space@yourService - EPFL (Swiss Federal Institute of Technology)** Jul 2020 – May 2021
- Achieved **92% detection accuracy** by building a **Python-based** moonquake detection system using signal processing and anomaly detection.
 - Reduced false positives by building a **multi-stage anomaly detection pipeline** using **bandpass filtering** on raw lunar rover seismic data.
 - Guaranteed **astronaut safety** by containerizing the detection system on **Docker** in coordination with EPFL for the Asclepius experiment.
 - Achieved **sub-5 second alert dispatch** by decoupling the detection engine from the notification layer via a **config-driven architecture**.

Technologies

Languages: Python, Go, C++, C, C#, Java, SQL, JavaScript, TypeScript, PHP, HTML, CSS, Unix Shell Scripting, Linux Kernel Programming
Frameworks & Libraries: Flask, Django, PyTorch, REST APIs, Node.js, Express, ReactJS, AngularJS, Bootstrap, Tomcat, BeautifulSoup
Tools: AWS, Docker, Kubernetes, Terraform, Jenkins, Ansible, Git, Bash, Datadog, PowerBI, PagerDuty, MongoDB, MySQL, PostgreSQL

Certifications

AWS Certified Solutions Architect – Associate, Amazon Web Services

Issued: 2023